7

Langage de manipulation de données

Langage SQL

Requête	Type langage	Définition (commandes)
SELECT	**LRD** Langage de récupération de données	récupérer des données contenues dans une ou plusieurs tables de la base
INSERT, UPDATE, DELETE, MERGE	**LMD** Langage de manipulation de données	Modifier les données de la base.
CREATE, ALTER, DROP, RENAME, TRUNCATE	**LDD** langage de définition de données	modifier la structure de la base.
COMMIT, ROLLBACK, SAVEPOINT	Ordre de contrôle de transaction	Administrer les changements effectués par les commandes DML
GRANT, REVOKE	**LCD** Langage de contrôle des données	contrôler les accès utilisateur à la base de données.

Requête INSERT

- **Cette requête permet d'insérer les données dans la table.**

Syntaxe:

INSERT INTO nom_de_table **(column1_name , column2_name, …)**

VALUES (value 1 , value1, …);

(column_name, column_name, …)	Permet de définir les colonnes dans lesquelles les valeurs seront insérées. Si l'insertion se fait dans toutes les colonnes cette clause sera retirée de l'ordre INSERT.
VALUES (value 1,value 2, …);	Permet de définir les valeurs à rajouter dans les colonnes de la table.

Création d'une table

Exercice

- Ecrire la requête qui insert un enregistrement dans la table « ***departement*** » avec les valeurs suivantes: **id_dep**=20 et **nom**='vente'

Enregistrement avec des valeurs nulles

- **Méthode explicite:**

  Spécifier le mot clé **NULL** dans la liste des valeurs.

  Exemple:

```
INSERT INTO departement (id_dep, nom) VALUES (20,NULL);
```

- **Méthode implicite:**

  Ne pas mettre la colonne dans la liste des champs

  Exemple:

```
INSERT INTO departement (id_dep) VALUES (20);
```

Autres méthodes d'insertion

- **Utilisation de fonctions:**

Différentes fonctions peuvent être utilisées lors de l'insertion.

Exemple:

```
INSERT INTO emp(id_emp, date_embauche) VALUES
(31,SYSDATE);
```

N.B: Les fonctions de formatage de données peuvent être utilisées TO_DATE, TO_CHAR ou TO_NUMBER

Autres méthodes d'insertion (2)

- **Copier des lignes depuis une autre table:**

  l'insertion se fera à l'aide d'une sous requête.

Exemple:

```
INSERT INTO departement (id_dep, nom) SELECT id, nom
FROM dep_siege WHERE id=14;
```

N.B:

- Le nombre de colonnes de la table doit être identique à celui de la requête SELECT.
- Le mot clé VALUES n'est pas utilisé.

Requête UPDATE

- Il est possible grâce à l'ordre **UPDATE** de modifier des valeurs de colonnes dans les tables.

Syntaxe:

```
UPDATE table
SET    column 1 = value 1, column2 = value 2, …
WHERE condition;
```

Exemple:

```
UPDATE emp
SET sal=sal*1.20
WHERE ename='Alaoui';
```

Requête UPDATE (2)

- **Remarque1:**

✓ Il est possible d'utiliser le mot clé **DEFAULT**.

Exemple:

```
UPDATE emp
SET date_embauche = DEFAULT
WHERE id_dep=1;
```

id_dep	Nom	Date_embauche
1	Karimi	12-jun-99
2	Hachimi	20-apr-99
3	Brahimi	1-mar-03

Requête UPDATE (3)

- **Remarque2:**

  ✓ En cas d'omission de la clause WHERE, toutes les lignes sont modifiées.

Exemple:

```
UPDATE emp
SET nom='karimi';
```

Id_dep	Nom	Date_embauche
1	Karimi	12-jun-99
2	Karimi	20-apr-99
3	Karimi	1-mar-03

Requête UPDATE (4)

- **Utilisation de sous requête:**

Effectuer des mises à jour de plusieurs colonnes à l'aide de sous requêtes.

Syntaxe:

```
UPDATE table
SET   colonne1 =(SELECT colonne1
                 FROM table
                 WHERE condition1)

      colonne2= (SELECT colonne2
                 FROM table
                 WHERE condition2)
[WHERE condition ];
```

Requête UPDATE (5)

- **Utilisation de sous requête (suite):**

Exemple:

```
UPDATE emp
SET   fonction=(SELECT fonction
                FROM emp
                WHERE id_emp=1)

      salaire= (SELECT salaire
                FROM emp
                WHERE id_emp=1)
WHERE nom='Alaoui';
```

Requête DELETE

- Pour effacer une ou plusieurs lignes d'une table, il suffit d'utiliser la commande **DELETE**

Syntaxe:

```
DELETE FROM table
WHERE condititon;
```

La commande **TRUNCATE** permet de vider une table.

Syntaxe:

```
TRUNCATE TABLE nom_table;
```

Requête DELETE (2)

- **Remarque:**

  En cas d'omission de la clause **WHERE**, toutes les lignes de la table seront supprimées. C'est équivaut à l'utilisation de la clause **TRUNCATE.**

Exemple:

DELETE FROM emp ;

Id_dep	Nom	salaire

Requête MERGE

- Insertion conditionnelle dans une table.
- Si la ligne existe dans la table, **MERGE** se comporte comme un **UPDATE**.
- Par contre si la ligne n'existe pas, **MERGE** se comporte comme un **INSERT**.

Syntaxe:

```
MERGE INTO table AS table_alias
USING  (table | vue | sous-requête) alias
ON (condition de jointure)
WHEN MATCHED THEN
        UPDATE SET  Col1 = col1_val1,
WHEN NOT MATCHED THEN
        INSERT (column_list)
        VALUES (column_values);
```

WHEN MATCHED THEN : si la ligne existe déjà, on fait la mise à jour.

Requête MERGE (2)

- **Exercice:**

Mettre à jour les enregistrements de la table « **copie_emp** » à partir de la table « **emp** » s'ils existent déjà et insérer les nouveaux enregistrements dans le cas contraire.

Solution:

```
MERGE INTO copie_emp AS c
USING emp e
ON (e.id_emp= c.id_emp)
WHEN MATCHED THEN
        UPDATE SET  c.nom=e.nom,
WHEN NOT MATCHED THEN
        INSERT (id_emp, nom)
        VALUES (e.id_emp, e.nom);
```

Manipulation des vues

Vues

Définition:

- Une vue est une table virtuelle qui représente le résultat d'une requête.

Rôle d'une vue

- Clarification et facilitation des requêtes;
- Limitation d'accès à des données dans la base de données;
- Réduction de la complexité syntaxique des requêtes;
- Définition des contraintes d'intégrité;
- Définition des schémas externes;

Vues (2)

Exemple:

Vols						
No_VoL	**No_AV**	**No_PIL**	**V_d**	**V_a**	**H_d**	**Ha**
V100	10	1	AL-Hoceima	Casa	8	10
V101	11	2	Fès	Agadir	10	12
V102	12	3	Casa	Oujda	17	20

Création d'une vue, nommée Ligne

Ligne	
V_d	**V_a**
AL-Hoceima	Casa
Fès	Agadir
Casa	Oujda

Vues (3)

Création d'une vue

CREATE [**OR REPLACE**] [**FORCE** | **NO FORCE**]
VIEW nom_table (liste_nom_colonnes)
AS requête **WHERE** (condition)
[**WITH CHECK OPTION | WITH READ ONLY**] **;**

RQ: **WITH CHECK OPTION**; empêche toutes les mises à jour de la Vue si les conditions de la clause WHERE ne sont pas respectées.

Exemple: vue correspond aux vols qui partent de la ville de **Fès**.

CREATE VIEW Vol_dep_Fès
AS SELECT No_VOL, V_a, H_d, H_a
FROM Vols
WHERE V_d = 'Fès' ;

Vues (4)

Interrogation

- Interrogation d'une vue

Interrogation de la vue correspondant aux vols qui partent de Fès

SELECT * **FROM** Vol_dep_Fès;

- Suppression d'une vue

DROP VIEW nom_vue ;

Suppression de la vue correspondant aux vols qui partent de Fès

- **DROP VIEW** Vol_dep_Fès;

Vues (5)

Exemple

- Vue correspondant les pilotes qui assurent le maximum de vols

```
CREATE VIEW Nbre_vols_par_pil
AS SELECT No_Pil, count(*) AS Nbr_vols
  FROM Vols WHERE No_PIL IS NOT NULL
GROUP BY No_PIL ;

SELECT No_Pil FROM Nbre_vols_par_pil
WHERE Nbr_vols = (SELECT max(Nbr_vols)
FROM Nbre_vols_par_pil) ;
```

Vues (6)

Mise à jour:

Quelques opérations sur les vues

- **INSERT**
- **DELETE**
- **UPDATE**

Limites:

Les instructions ne peuvent pas être appliquées aux vues contenant:

- L'un des opérateurs: **UNION**, **INTERSECT**, **MINUS**
- Une jointure
- La clause **DISTINCT**, pour une pseudo-colonne ou une expression dans la liste de sélection des colonnes.
- Une clause **GROUP BY**, **ORDER BY,**

Vues (7)

Exemples:

- Création de la vue correspondante à la personne définissant les vols

```
CREATE VIEW def_vols
AS SELECT No_VOL, V_d, D_d, V_a, D_a    //D_d: date de départ
FROM vols
WHERE No_VOL IS NULL AND No_PIL IS NULL ;
```

- Définition d'un nouveau vol

```
INSERT INTO def_vols VALUES
('V104', Marrakech', to date('10/03/18 11 :00', 'DD/MM/RR HH :MI'),
'Rabat', to date('10/03/18 13 :00', 'DD/MM/RR HH :MI')) ;
```

Vues (8)

Exemples (suite)

- Suppression d'un vol non affecté

  DELETE FROM def_vols

  WHERE No_VOL = 'V103' ;

- Modification d'un vol non affecté

  UPDATE def_vols

  SET D_d= D_d + 1/24, D_a= D_a + 1/24 **WHERE** No_VOL = 'V103' ;

- Connaissance des vols non affectés

  SELECT * FROM def_vols **;**

Vues (9)

Exemples (suite)

- Création de la vue correspondante à la personne affectant un avion et un pilote à un vol

  CREATE VIEW affect_vols

  AS SELECT No_VOL, No_AV, No_PIL **FROM** vols **;**

- Affectation d'un avion et un pilote à un nouveau vol

  UPDATE affect_vols

  SET No_AV = 11, No_PIL = 4

  WHERE No_VOL='V104'**;**

- Affectation d'un nouveau pilote à un vol

  UPDATE affect_vols

  SET No_PIL = 5 **WHERE** No_VOL = 'V104' ;

Vues (10)

Contrôle de mise à jour

- Création d'une vue de vérification : contrôle d'insertion ou de modification de ligne

  CREATE VIEW nom_vue

  AS requête

  WITH CHECK OPTION;

- Vérification des contraintes: interdiction des valeurs inconnues.

  CREATE VIEW v_vols

  AS SELECT * FROM Vols

  WHERE No_PIL> 0 **AND** No_AV > 0

  AND V_d **IN** ('Fès', 'AL-Hoveima', 'Casa')

  WITH CHECK OPTION;

Vues (11)

Contraintes de référence

- Validation d'insertion des données se fait dans la table référençant.
- Validation de la suppression des données se fait dans la table référencée.

Règle d'adéquation : les insertions et les suppressions se font toujours à travers des vues

Exemple : expression de clés étrangères de la relation **Vols**

- Validation des insertions dans « **Vols** »

CREATE VIEW av_ avions **AS SELECT** * **FROM** **Vols**

WHERE No_AV > 0 **AND** (No_PIL **IS NULL OR** No_PIL **IN(SELECT** No_PIL **FROM** Pilotes)) **AND** (NOM_AV **IS NULL OR** NOM_AV **IN** (SELECT NOM_AV **FROM** Avions))

WITH CHECK OPTION; // Trois tables: Vols, Pilotes et Avion

Vues (12)

Exemple (suite): expression de clés étrangères de la relation **Vols**

- Validation des suppressions dans « **Avions** » et « **Pilotes** »

```
CREATE VIEW sup_avions
AS SELECT * FROM Avions A
WHERE NOT EXISTS ( SELECT * FROM Vols V
WHERE A.No_AV = V.No_AV) ;

CREATE VIEW sup_pilotes
AS SELECT * FROM Pilotes P
WHERE NOT EXISTS ( SELECT * FROM Vols V
WHERE P.No_PIL = V.No_PIL) ;
```